

Documento de Especificação Técnica (SDD)

Projeto: Lousa Digital Interativa (PWA & Desktop)

Resumo Executivo: Este documento estabelece as especificações arquiteturais e de comportamento para a Lousa Digital Interativa, operável como PWA e Desktop via Electron, empregando Vanilla JS e Canvas API para máxima performance.

1. Escopo e Objetivos do Sistema

O objetivo principal é construir uma lousa digital modular, de alta performance, sem dependências de frameworks pesados, projetada especificamente para fins educacionais, apresentações e aulas online.

Requisitos de Negócio

- **Portabilidade Absoluta:** O mesmo código base do frontend deve rodar perfeitamente no navegador (online e offline) e empacotado como executável desktop.
- **Interatividade Multimídia:** Permitir a exibição paralela de materiais de apoio (PDFs, vídeos locais e vídeos do YouTube) sem obstruir ou poluir a área de desenho principal da lousa.
- **Auto-Suficiência:** Capacidade de gravar a aula (tela e áudio do microfone) e salvar o progresso localmente de forma nativa e segura.

2. Arquitetura e Estrutura de Diretórios

A aplicação adota uma separação rigorosa entre o processo de ciclo de vida do sistema operacional (Electron) e a camada de interface/regra de negócio (Renderer), garantindo isolamento e segurança.

```
/lousa-digital
├── package.json      # Dependências do Node.js e scripts
├── main.js           # Processo Principal do Electron
├── preload.js        # Ponte de segurança IPC
└── /src              # Raiz do Frontend (Renderer / PWA)
    ├── index.html    # Estrutura HTML única (SPA)
```

```

├─ manifest.json      # Configuração do PWA
├─ sw.js              # Service Worker (Cache Offline)
├─ /css
│   └─ style.css      # Estilização global
│   └─ modals.css     # Estilização de modais
├─ /js
│   └─ /core
│       └─ canvas-engine.js # Motor 2D e pincéis
│       └─ history.js      # Histórico (Undo/Redo)
│   └─ /modules
│       └─ ui-manager.js   # Controle da interface
│       └─ media-viewer.js # Modais (YouTube, Vídeo, PDF)
│       └─ recorder.js     # Captura de mídia
│   └─ /services
│       └─ storage.js      # Persistência do IndexedDB
└─ /assets              # Ícones e texturas

```

3. Especificações dos Módulos Core

3.1 Motor do Canvas (`canvas-engine.js`)

Responsável pela captura de entradas analógicas/digitais e pela renderização síncrona dos traços.

- **Tecnologia:** HTML5 Canvas API (Contexto `2d`).
- **Eventos:** Uso de **Pointer Events** (`pointerdown` , `pointermove` , `pointerup`) para suporte unificado a mouse, touch e canetas stylus com pressão.
- **Suavização:** Interpolação por curvas quadráticas de Bézier (`quadraticCurveTo`) para suavizar curvas desenhadas rapidamente.
- **Borracha:** Configuração `globalCompositeOperation = 'destination-out'` para apagar preservando o fundo (texturas/cores).

3.2 Gerenciador de Mídia e Modais (`media-viewer.js`)

- **Interface:** Utilização da tag `<dialog>` nativa do HTML5.
- **PDFs:** Integração da biblioteca `pdf.js` para renderizar as páginas em um canvas secundário no modal.
- **Vídeos:** Tag `<video>` para arquivos locais e `<iframe>` para links do YouTube.

3.3 Persistência de Dados (`storage.js`)

Armazenamento em **IndexedDB** para salvar as notas sem limites rígidos de cota.

```
{
  "id": 1,
  "title": "Aula de Matemática",
  "backgroundColor": "#123424",
  "strokes": [
    {
      "tool": "brush",
      "color": "#ffffff",
      "lineWidth": 4,
      "points": [ { "x": 120, "y": 200, "pressure": 0.6 } ]
    }
  ]
}
```

3.4 Gravador de Tela e Áudio (`recorder.js`)

- **Tecnologia:** `MediaRecorder API`.
- **Captura:** `getDisplayMedia()` (tela) + `getUserMedia()` (microfone).
- **Saída:** Geração de um arquivo `.webm` consolidado para download.

4. Critérios de Aceite (Testes)

MÓDULO	AÇÃO DE TESTE	RESULTADO ESPERADO
Canvas	Desenhar com caneta stylus.	O traço varia a espessura de acordo com a pressão.
Canvas	Apagar traço existente.	Traço é removido expondo a textura/cor de fundo.
Modais	Abrir arquivo PDF.	Renderização interna via <code>pdf.js</code> sem sair da lousa.
Storage	Fechar e reabrir app.	Sessão atual é recuperada automaticamente (IndexedDB).
Recorder	Gravar aula.	Arquivo de vídeo e áudio síncronos é baixado com sucesso.